Propuesta – Proyecto Final Bases de Datos 2: Tienda de Ropa

Oct 26, 2026 · @Juan Mateus

Datos de identificación

Campo	Detalle
Universidad	Universidad El Bosque
Programa	Ingeniería de Sistemas
Asignatura	Bases de Datos 2 (IS000101-02-0015), grupo G3
Docente	Ing., Esp., M.Ed. Christian Felipe Duarte
Integrantes	Juan Esteban Mateus · Diego Lemus
Proyecto	Tienda de Ropa: sistema web de ventas, inventario y fidelización
Repositorio	github.com/Jmateus04/tienda_ropa_bd2
Entrega final	11 de noviembre de 2026
Sustentación	20 de noviembre de 2026

Planteamiento del problema

En las tiendas de ropa de cadena, el inventario que muestra el sistema no coincide con lo que realmente se puede vender. El origen principal es que las unidades averiadas siguen contadas como disponibles.

Unidades averiadas que el sistema reporta como vendibles. Una prenda manchada, rota o sin botón sigue sumando en el stock de su talla. Si hay 4 unidades de talla S y una está averiada, el sistema reporta 4. El asesor ofrece una prenda que no existe en condición de venta, el cliente espera, la búsqueda falla y la venta se pierde.

Tallas agotadas sin aviso. El stock se maneja por producto y no por la combinación de talla y color. Una referencia puede figurar con existencias mientras su talla más vendida ya se acabó. Nadie se entera hasta que un cliente la pide.

Inventario sin trazabilidad. No queda un registro claro de por qué cambió el stock: una venta, una devolución, una avería o una entrada de mercancía. Esto vuelve lento explicar los descuadres en los conteos físicos.

Beneficios a clientes aplicados a mano. Las tiendas manejan dos tipos de beneficio:

- Bonos por monto de compra, por ejemplo $70.000 de descuento en compras desde $300.000.
- Un descuento en el mes de cumpleaños que crece con la fidelidad del cliente: 20%, 25% o 30%.

Cuando el asesor decide cuál aplica, el resultado depende de quién atienda. Puede regalar de más o quedarse corto, y la tienda no sabe cuánto le cuestan sus beneficios.

Pregunta del proyecto: ¿cómo diseñar una base de datos que garantice que solo se venda lo que realmente está disponible, que registre cada movimiento del inventario y que aplique los beneficios de forma automática y consistente?

Justificación

Los problemas descritos se resuelven mejor dentro del motor de base de datos que en la aplicación. Una regla que vive en la base se cumple sin importar qué pantalla, script o usuario modifique los datos.

- **Integridad del inventario.** Las restricciones CHECK y los triggers impiden vender unidades averiadas o dejar stock negativo, aunque alguien intente hacerlo por fuera de la aplicación.
- **Atomicidad.** Una venta toca varias tablas a la vez: venta, detalle, pagos, inventario y fidelidad del cliente. Un procedimiento con control de transacciones garantiza que se registre todo o nada; una venta a medias descuadraría el inventario.
- **Concurrencia.** Dos cajas pueden intentar vender la última unidad de una talla al mismo tiempo. El bloqueo de la fila de la variante dentro de la transacción evita la doble venta.
- **Trazabilidad.** Un trigger registra cada cambio de stock con su tipo y su fecha, sin depender de que el programador lo recuerde.
- **Reglas de negocio consistentes.** Una sola función calcula el descuento aplicable, así que todas las cajas aplican la misma regla.
- **Datos con estructura variable en MongoDB.** Las reseñas, los carritos abandonados y las fichas técnicas cambian de forma según el producto. Un modelo documental los guarda sin forzar columnas vacías en el modelo relacional.

El proyecto aplica así los contenidos de Bases de Datos 1 y 2 a un caso que uno de los integrantes conoció de primera mano trabajando en una tienda de ropa.

Objetivos

Objetivo general

Desarrollar un sistema de información web para una tienda de ropa. La lógica de ventas, inventario y fidelización se implementa en PostgreSQL con PL/pgSQL, integrado con MongoDB, desplegado en una máquina virtual Linux y gestionado como un proyecto formal de TI.

Objetivos específicos

1. Diseñar el modelo entidad-relación y el modelo relacional del negocio, con al menos 8 entidades, relaciones 1:1, 1:N y N:M, y normalización hasta tercera forma normal.
2. Implementar en PostgreSQL funciones, procedimientos almacenados y triggers que controlen el inventario por talla y color, separando las unidades disponibles de las averiadas.
3. Garantizar la atomicidad de las ventas mediante control de transacciones (COMMIT y ROLLBACK) y manejo de excepciones en la capa de datos.
4. Automatizar el cálculo de descuentos de fidelización: bono por monto mínimo de compra o descuento de cumpleaños según el nivel del cliente, aplicando el que más favorezca al cliente.
5. Definir roles y permisos de base de datos (cajero, administrador y analista) para controlar quién puede consultar y modificar cada objeto.
6. Diseñar e implementar en MongoDB las colecciones de reseñas, carritos abandonados y fichas técnicas, con operaciones CRUD, consultas complejas y agregaciones.
7. Construir una aplicación web en Node.js con Express conectada a ambas bases de datos, que invoque los procedimientos almacenados.
8. Desplegar la solución en una máquina virtual con Ubuntu Server y documentar su instalación y configuración.
9. Gestionar el proyecto con cronograma, matriz RACI, matriz de riesgos, metodología Kanban y control de versiones en GitHub.

Alcance

El sistema cubre la operación de una sola tienda física: catálogo, inventario por variante, ventas en caja y fidelización de clientes.

Incluye	No incluye
Catálogo de productos con categorías, tallas, colores y variantes (SKU)	Varias sedes o bodegas, y traslados entre ellas
Inventario por variante con stock disponible y stock averiado separados	Facturación electrónica ante la DIAN
Registro de averías, entradas de mercancía y movimientos de inventario	Pasarela de pagos real: los pagos se registran, no se cobran
Ventas con varios métodos de pago y anulación de ventas	Tienda en línea para clientes finales
Clientes, niveles de fidelidad, promociones y cálculo automático de descuentos	Nómina, contabilidad y cuentas por pagar a proveedores
Proveedores y el costo de cada producto por proveedor	Aplicación móvil
Roles de base de datos: cajero, administrador y analista	Alta disponibilidad, réplicas o despliegue en la nube
Reportes gerenciales en SQL	
Reseñas, carritos abandonados y fichas técnicas en MongoDB	
Aplicación web de demostración conectada a PostgreSQL y MongoDB	
Máquina virtual Ubuntu Server con ambos motores instalados	

Descripción de la solución

El sistema se organiza en cinco módulos. Cada regla de negocio crítica está implementada en la base de datos, no en la aplicación.

Módulo	Qué hace	Objetos de base de datos principales
Catálogo	Productos, categorías, tallas, colores, variantes (SKU) y proveedores	Tablas CATEGORIA, PRODUCTO, TALLA, COLOR, VARIANTE_PRODUCTO, PROVEEDOR, PRODUCTO_PROVEEDOR

Módulo	Qué hace	Objetos de base de datos principales
Inventario	Entradas de mercancía, registro de averías y auditoría de cada cambio de stock	`sp_entrada_inventario`, `sp_registrar_averia`, `fn_stock_disponible`, trigger de auditoría y de alerta de stock mínimo
Ventas	Registro y anulación de ventas, varios pagos por venta	`sp_registrar_venta`, `sp_anular_venta`, trigger que impide stock negativo
Fidelización	Niveles de cliente, promociones y cálculo del descuento	`fn_calcular_descuento`, trigger `trg_actualizar_nivel`
Reportes y documentos	Informes gerenciales en SQL; reseñas, carritos y fichas en MongoDB	Consultas con CTE y funciones analíticas; agregaciones en MongoDB

Reglas de negocio

1. **Solo se vende stock disponible.** Cada variante guarda `stock_disponible` y `stock_averiado` por separado. Una venta solo descuenta del disponible; si no alcanza, la transacción completa se revierte.
2. **Registro de averías.** Al registrar una avería, las unidades pasan de disponible a averiado en una sola transacción, con el motivo de la avería.
3. **Todo cambio de stock queda registrado.** Cada venta, anulación, entrada o avería genera un movimiento de inventario con su tipo, cantidad y fecha.
4. **Venta atómica.** El registro de la venta, sus detalles, sus pagos, el descuento del inventario y la actualización de la fidelidad del cliente ocurren juntos o no ocurren.
5. **Niveles de fidelidad.** El nivel depende de lo comprado en los últimos 12 meses y se recalcula después de cada venta:

Nivel	Compras en 12 meses	Descuento en el mes de cumpleaños
Clásico	Cualquier cliente registrado	20%
Frecuente	Desde $1.500.000	25%
VIP	Desde $4.000.000	30%

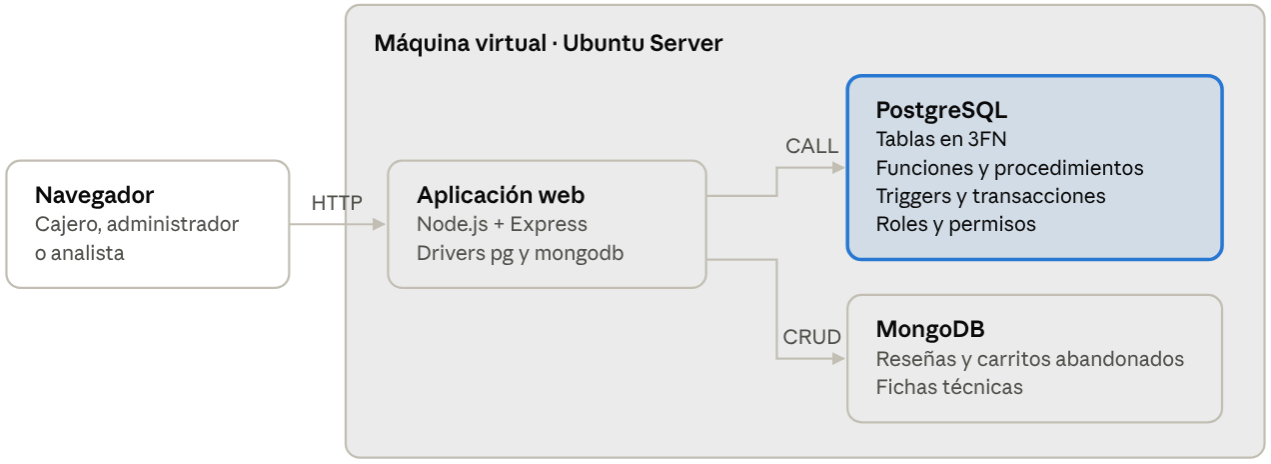
6. **Bono por monto de compra.** Una promoción activa entrega un bono fijo cuando el subtotal alcanza un mínimo, por ejemplo $70.000 en compras desde $300.000. La tienda puede crear promociones nuevas sin cambiar el código.

7. **Descuentos no acumulables.** Si un cliente califica a la vez para el descuento de cumpleaños y para un bono, el sistema aplica solo el de mayor valor para el cliente.
8. **Alerta de stock mínimo.** Cuando el stock disponible de una variante baja de su mínimo, se genera una alerta para reponerla.

Arquitectura y tecnologías

La solución es un monolito web de tres capas desplegado en una sola máquina virtual. La aplicación no calcula reglas de negocio: llama a los procedimientos de PostgreSQL y consulta MongoDB para los datos documentales.

Las reglas de negocio viven en PostgreSQL; la aplicación solo las invoca



arquitectura · 3 componentes en una VM

Componente	Tecnología	Por qué
Base relacional	PostgreSQL	PL/pgSQL es el lenguaje más cercano a PL/SQL de Oracle visto en clase; los procedimientos admiten COMMIT y ROLLBACK, y los bloques EXCEPTION manejan errores
Base documental	MongoDB	Obligatoria en el enunciado; ajusta a documentos de estructura variable
Aplicación	Node.js + Express	Un solo lenguaje en servidor y vistas, con drivers maduros para ambas bases (`pg` y `mongodb`)
Sistema operativo	Ubuntu Server LTS	Distribución Linux con soporte largo y abundante documentación para PostgreSQL y MongoDB

Componente	Tecnología	Por qué
Virtualización	VirtualBox	Gratuito y disponible en los equipos de ambos integrantes
Modelado	MySQL Workbench o pgModeler	Diagramas en notación Crow's Foot exigidos en la Fase 1
Control de versiones	Git + GitHub	Ramas, Pull Requests y tablero Kanban en GitHub Projects

Metodología y cronograma general

El proyecto se gestiona con **Kanban**: el equipo es de dos personas, el plazo es de 16 días y el trabajo fluye por tareas pequeñas, sin sprints fijos ni ceremonias que no aportan a esa escala. El tablero vive en GitHub Projects con cinco columnas (Backlog, Por hacer, En progreso, En revisión, Hecho). Cada tarea se integra a la rama `develop` mediante un Pull Request revisado por el otro integrante.

Hito	Fecha	Entregable
H1. Propuesta y supuestos	27 oct 2026	Documento de propuesta y documento de supuestos
H2. Diseño de datos	30 oct 2026	Modelos E-R (Chen y Crow's Foot), modelo relacional, diccionario de datos
H3. Gestión del proyecto	30 oct 2026	Requerimientos, Gantt, RACI, matriz de riesgos
H4. Infraestructura	1 nov 2026	VM Ubuntu Server con PostgreSQL y MongoDB, análisis de SO y RDBMS
H5. Lógica en base de datos	5 nov 2026	Scripts de creación e inserción, funciones, procedimientos, triggers, roles y reportes; MongoDB con CRUD y agregaciones
H6. Aplicación integrada	8 nov 2026	Aplicación web conectada a PostgreSQL y MongoDB
H7. Cierre	10 nov 2026	Informe ejecutivo, referencias y video demostrativo
Entrega final	11 nov 2026	Repositorio y entregables completos

Hito	Fecha	Entregable
Sustentación	20 nov 2026	Presentación ante el docente